

## Overview

Multi-agent systems partition complex tasks across specialized agents. Instead of one generalist agent doing everything, a team of agents handles decomposition, execution, critique, and synthesis.

A formal task decomposition view:

$$G = (V, E), \quad V = \{\text{tasks}\}, \quad E = \{\text{dependencies}\}$$

Each task node can be assigned to an agent role. The orchestration layer enforces order, retries, and merge behavior.

Why teams adopt multi-agent systems:

- specialization by skill
- parallel execution of independent subtasks
- modular testing and replacement

Why they can fail:

- coordination errors
- message ambiguity
- cost explosion from redundant calls

## Multi-Agent Design Patterns

### Coordinator-Worker Pattern

- Coordinator receives objective and decomposes work.
- Worker agents perform specialized subtasks.
- Coordinator aggregates outputs and resolves conflicts.

Best for predictable workflows such as support triage or report generation.

### Peer-to-Peer Collaboration

Agents exchange messages without central control. This improves flexibility but can create deadlocks or repetitive loops if no arbitration exists.

### Hierarchical Teams

- Executive planner -> domain leads -> executors.
- Mirrors organizational structures.
- Supports large systems where one global planner is insufficient.

### Critic-Generator Pattern

One agent proposes output, another evaluates against rubric, optional third revises. Useful for quality-sensitive tasks such as policy documents.

## Workflow & Orchestration Concepts

### Task and Dependency Modeling

Define each task with:

- input contract

- output schema
- completion criteria
- timeout and retry policy

Dependency types:

1. strict sequence (A then B)
2. fan-out/fan-in parallel branches
3. conditional branching based on intermediate state

## Orchestration vs Choreography

- Orchestration: central controller enforces workflow.
- Choreography: agents coordinate through events and shared protocols.

Pragmatic guidance:

- Use orchestration for reliability and compliance.
- Use choreography for highly dynamic, distributed ecosystems.

## Tools & Frameworks

### LangChain

Useful for composing multi-step chains and tool-calling pipelines. Good entry point for structured but lightweight workflows.

### LangGraph

State graph abstraction for agent workflows with nodes, edges, conditional routing, checkpoints, and interrupt points. Useful when you need deterministic control over complex loops.

### CrewAI

Role-based multi-agent framework centered on agents, tasks, crews, and flow control. Emphasizes collaborative teams and production readiness features.

### AutoGen

Event-driven, asynchronous multi-agent patterns with strong support for distributed systems and message-based collaboration.

Framework decision heuristic:

- Need explicit graph state and advanced control flow -> LangGraph.
- Need role-based teamwork semantics quickly -> CrewAI.
- Need distributed event-driven agent mesh -> AutoGen.

## Failure Modes & Robustness

### Common Failure Modes

1. Deadlock
  - Agents wait on each other indefinitely.
2. Oscillation
  - Agents repeatedly revise without convergence.
3. Conflict

- Agents produce incompatible outputs.
4. Tool thrashing
- Excessive repeated tool calls increase latency and cost.

## Guardrails

- Maximum iteration budgets.
- Typed message schemas.
- Arbitration policy for conflicting outputs.
- Human escalation rules.
- Circuit breakers for runaway loops.

## Evaluation Metrics

- End-task success rate.
- Cost per completed task.
- Mean steps to completion.
- Conflict resolution rate.
- Human override frequency.

## Implementation Blueprint

A robust multi-agent orchestration pipeline usually has:

1. Request router.
2. Task graph builder.
3. Agent runtime (state + messaging).
4. Tool gateway with policy checks.
5. Aggregator/verifier.
6. Trace and evaluation store.

Text diagram:

Frontend -> Coordinator -> Worker Agents -> Tool Layer -> Aggregator -> User

## Business Case Studies & Exceptions

### Case Study 1: Multi-Agent Customer Support

Roles:

- Triage agent classifies intent and urgency.
- Knowledge agent retrieves policy/context.
- Response agent drafts answer.
- Escalation agent routes complex/high-risk cases.

Impact:

- Faster handling for repetitive tickets.
- Better consistency in response format.

Exception:

- For low-volume teams, a single-agent tool router may be cheaper and easier to maintain.

## Case Study 2: Data Pipeline Agent Team

Roles:

- ETL agent runs extraction and transformation.
- QA agent validates schema and quality checks.
- Reporting agent generates stakeholder summary.

Benefits:

- Clear ownership per stage.
- Easier root-cause analysis when failures occur.

Risk:

- Without strict contracts, downstream agents ingest malformed outputs.

## Interview Questions & Answers

1. **Why use multi-agent systems instead of one agent?**  
To partition complexity across specialized roles and optionally parallelize work.
2. **Explain coordinator-worker pattern.**  
A central controller decomposes tasks, delegates to workers, then aggregates results.
3. **What is orchestration vs choreography?**  
Orchestration uses central control; choreography relies on decentralized event coordination.
4. **What is a deadlock in multi-agent workflows?**  
A state where agents wait indefinitely on each other and no progress is possible.
5. **How do you prevent oscillation loops?**  
Set iteration limits, convergence criteria, and explicit termination rules.
6. **When is hierarchical design useful?**  
Large workflows with multiple domains where one coordinator becomes a bottleneck.
7. **How do you resolve conflicting agent outputs?**  
Use arbitration rules, confidence scoring, or a dedicated verifier agent.
8. **What metrics matter most?**  
Task success, cost, latency, override rate, and policy compliance.
9. **How does LangGraph help reliability?**  
It models state transitions explicitly with controlled routing and checkpoints.
10. **What does CrewAI emphasize?**  
Role-based collaborative crews and flow orchestration primitives.
11. **What does AutoGen emphasize?**  
Asynchronous, event-driven, distributed agent communication.
12. **What is tool thrashing?**  
Repeated unnecessary tool calls caused by weak planner policies.
13. **How do you secure multi-agent systems?**  
Least-privilege tools, signed actions, policy checks, and auditable traces.
14. **Can multi-agent improve quality?**  
Yes, especially with critic-verifier loops, but only if coordination overhead is controlled.
15. **When should you simplify architecture?**  
If reliability gains are small relative to operational complexity and cost.

## References

- LangGraph docs: <https://docs.langchain.com/oss/python/langgraph/graph-api> and <https://docs.langchain.com/oss/python/langgraph/agents>
- CrewAI docs: <https://docs.crewai.com/>
- AutoGen user guide: <https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/index.html>
- Anthropic effective agents guide: <https://resources.anthropic.com/building-effective-ai-agents>
- DeepLearning.AI Agentic AI course page: <https://www.deeplearning.ai/courses/agentic-ai>
- DivVerse curriculum page: <https://divverse.com/agentic-ai-course-curriculum/>
- Scaler agentic AI syllabus: <https://www.scaler.com/blog/agentic-ai-syllabus/>